Polyglot Import CSV: Uma solução para importação de documentos CSV para múltiplos bancos de dados

Lucas Bueno Cesario

2023

RESUMO

A persistência poliglota é essencial na arquitetura contemporânea de bancos de dados, integrando diversas tecnologias de armazenamento em aplicativos. A ferramenta "Polyglot Import CSV" destaca-se como solução para otimizar a importação de dados CSV em diversos bancos de dados, superando desafios da persistência poliglota. Por meio de configurações simples em arquivos JSON, a ferramenta oferece suporte para PostgreSQL, Redis, Cassandra, MongoDB e Neo4j, acelerando o processo e aumentando a flexibilidade para os desenvolvedores. A publicação do código aberto da ferramenta impulsiona avanços colaborativos na dinâmica arquitetura moderna de bancos de dados, consolidando-se como uma solução versátil e eficiente.

Palavras-chave: Persistência poliglota; Ferramenta "Polyglot Import CSV"; Importação de dados CSV; JSON; SQL; NoSQL; Software Livre.

1 INTRODUÇÃO

1.1 Visão Geral

Nas esferas industrial e acadêmica de bancos de dados, a concepção de que um único modelo de dados não atende a todas as necessidades tem conquistado aceitação, apontando para a perspectiva de persistência poliglota no futuro. Embora os sistemas de banco de dados relacionais devam manter sua predominância, espera-se uma convivência com diversas formas de sistemas, como os NoSQL (Sadalage; Fowler, 2013).

Algumas pesquisas têm sido feitas na área de persistência poliglota, como sugestões de modelos de dados unificados (Chillón et al., 2023), comparação de desempenho de

bancos de dados multi-modelos contra abordagens de persistência poliglota, demonstrando que a persistência poliglota traz mais confiabilidade nas operações (Ye et al., 2023), e até mesmo a adoção da persistência poliglota em Big Data (Khine; Wang, 2019).

Percebe-se a falta de uma ferramenta única de importação de dados que possa realizar a persistência poliglota para diversos SGBDs que tratem modelos de dados diferentes. A proposta deste trabalho é apresentar a ferramenta *Polyglot Import CSV*, que importa dados armazenados em CSV para diferentes sistemas, baseada em uma configuração em JSON que aponta quais colunas do CSV formam cada entidade e relacionamento de cada SGBD.

CSV (Comma-Separated Values) é um arquivo de valores separados por vírgulas, frequentemente visualizado no Excel ou em alguma outra ferramenta de planilha. Pode haver outros tipos de valores como delimitadores, mas o mais comum é a vírgula. Muitos sistemas e processos hoje convertem seus dados para o formato CSV para saídas de arquivo para outros sistemas, relatórios amigáveis para humanos e outras necessidades. É um formato de arquivo padrão com o qual humanos e sistemas já estão familiarizados ao usar e manipular.

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver a ferramenta *Polyglot Import CSV* que possibilitará a importação de dados em arquivos CSV para múltiplos SGBDs: PostgreSQL, Redis, MongoDB, Cassandra ou Neo4j.

1.2.2 Objetivos Específicos

- Fazer com que o arquivo de configuração JSON seja bem personalizável, seguindo uma sintaxe específica, porém flexível, além de adicionar opções como filtros e escolha de chave primária;
- Não permitir sintaxes de configuração incorretas; se a sintaxe estiver errada em algum lugar do arquivo de configuração, a importação não deve começar até que o arquivo de configuração esteja corrigido;
- Testar a ferramenta com um conjunto de dados armazenados num arquivo CSV que representa dados de um site de e-commerce.

1.3 Metodologia

Buscando obter conhecimento para realização do trabalho, primeiramente foi feito um estudo básico sobre o assunto, seguindo da implementação da solução, como descrito abaixo:

- Estudo sobre persistência poliglota e seu estado da arte;
- Estudo sobre os paradigmas relacional e não-relacionais que serão abrangidos pela solução, tais como: chave-valor, documento, grafo e colunar;
- Estudo de decisão de linguagem de implementação que demonstra melhor produtividade para codificar a solução;
- Implementação da solução "Polyglot Import CSV";
- Testes da solução "Polyglot Import CSV" implementada.

Na primeira etapa, foi estudado o que é a persistência poliglota, e a sua relevância no cenário atual.

Na segunda etapa, foram estudados os diversos modelos de dados: como cada modelo representa suas entidades; se há relacionamentos entre entidades no modelo; se é possível a existência de entidades aninhadas e multivaloradas; as limitações de cada modelo.

Na terceira etapa, foram estudadas as formas de implementar a solução usando as linguagens Java e Python e, pela simplicidade e produtividade, a linguagem Python foi escolhida.

As últimas etapas, que estão em andamento, se tratam da implementação da solução proposta, "Polyglot Import CSV", e dos testes da mesma usando um arquivo CSV criado manualmente.

1.4 Estrutura do Trabalho

Este trabalho está estruturado em cinco capítulos principais. O primeiro aborda os objetivos, a metodologia e o contexto que levaram à elaboração deste TCC. O capítulo 2 revisa fundamentos teóricos. O capítulo 3 apresenta trabalhos relacionados. O capítulo 4 descreve a proposta da ferramenta *Polyglot Import CSV* e o capítulo 5 as atividades futuras (TCC II).

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Persistência Poliglota

Em 2008, Neal Ford, em seu livro "The Productive Programmer", introduziu o conceito de "programação poliglota" para expressar a ideia de que aplicativos devem ser desenvolvidos utilizando uma combinação de linguagens, aproveitando o fato de que diferentes linguagens são mais adequadas para lidar com diferentes problemas.

Uma das consequências notáveis da programação poliglota é a transição para a "persistência poliglota", usando a mesma analogia: pode-se dizer que múltiplas tecno-

logias de bancos de dados podem ser utilizados como armazenamento persistente para diferentes tipos de casos de uso. Embora ainda exista uma quantidade considerável de dados gerenciados em sistemas relacionais, a abordagem predominante passa a ser a reflexão sobre como se deseja manipular os dados antes de determinar qual tecnologia é a mais adequada para essa manipulação (Fowler, 2011).

A persistência poliglota abrange uma variedade de formatos de dados, incluindo estruturados, semi-estruturados e não estruturados. Especificamente, dados estruturados geralmente englobam formatos como relacional, chave-valor e em grafo. Dados semi-estruturados incluem principalmente documentos JSON e XML. Por fim, dados não estruturados são representados por arquivos de texto, bem como dados de bancos colunares (Ye et al., 2023).

A seguir, há um exemplo de aplicação prática da persistência poliglota em um e-commerce:

- **Dados Financeiros/Transacionais de Pagamento e Inventário de Produtos:** Esses dados podem ser armazenados em um banco de dados relacional, como o PostgreSQL.
- **Dados do Catálogo de Produtos:** Esses dados podem ser armazenados em um banco de dados NoSQL baseado em documentos, como o MongoDB.
- **Dados da Sessão do Carrinho de Compras e do Usuário:** Esses dados podem ser armazenados em um banco de dados NoSQL baseado em chave/valor, como o Redis.
- **Dados de Atividade do Usuário:** Estes dados podem ser armazenados em bancos de dados NoSQL colunares como o Apache Cassandra.
- **Dados de Recomendação:** Esses dados de recomendação podem ser armazenados em um banco de dados NoSQL baseado em grafos, como o Neo4j.

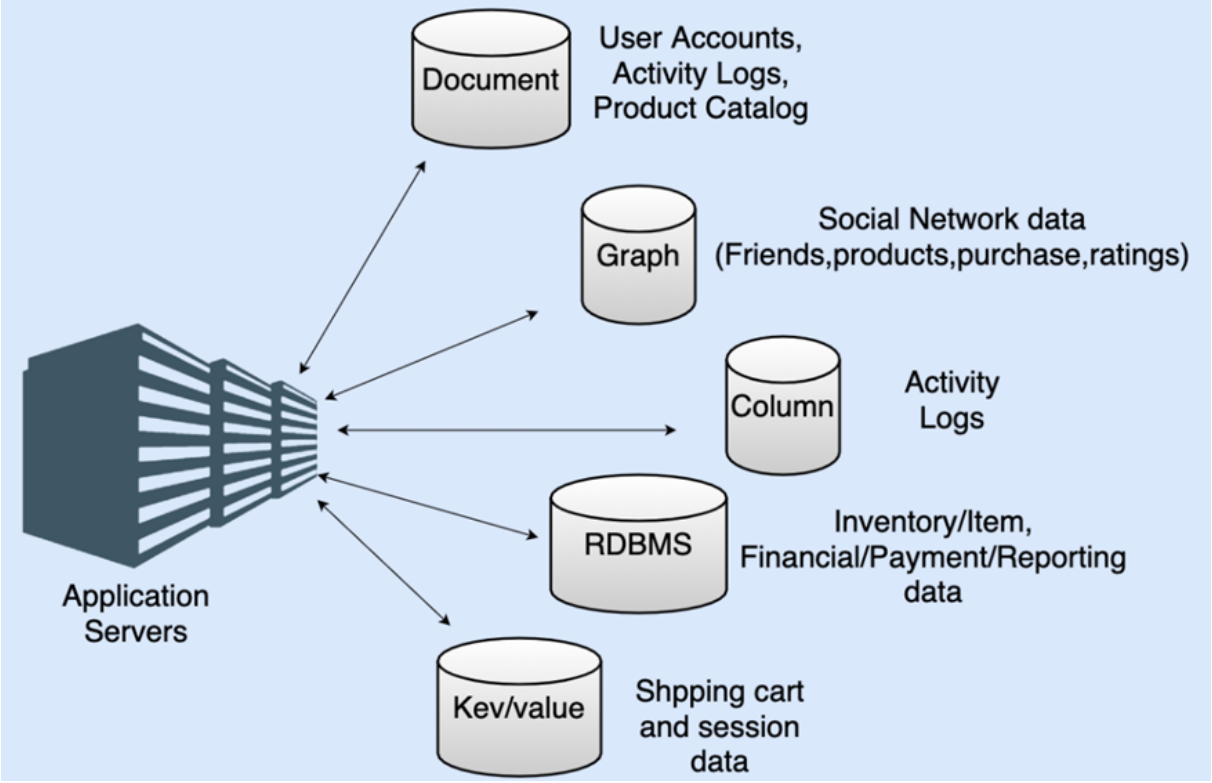

Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce.

2.2 Bancos de Dados NoSQL

Os bancos de dados NoSQL podem ser divididos em subcategorias com base em seus modelos de dados. Este trabalho utiliza a classificação de Hecht e Jablonski (Hecht; Jablonski, 2011), que categoriza os bancos de dados NoSQL em quatro tipos principais: chave-valor, colunas, documentos e grafos.

A Figura 2 ilustra esses modelos, de forma respectiva:

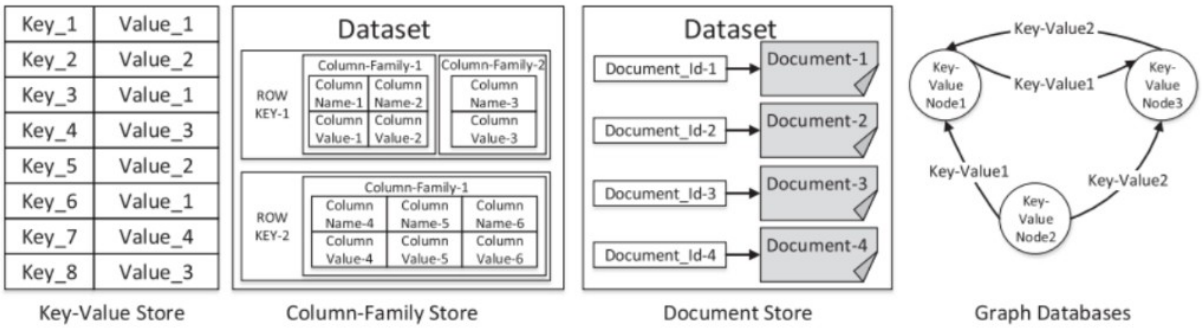

Figura 2 – Diferentes tipos de modelos de dados NoSQL.

2.2.1 Chave-valor

Os bancos de dados de chave-valor têm um modelo de dados simples baseado em pares de chave-valor, semelhante a um mapa associativo ou dicionário. A chave identifica exclusivamente o valor e é usada para armazenar e recuperar o valor no banco de dados, funcionando como uma chave primária. O valor pode ser usado para armazenar qualquer dado arbitrário, incluindo um número inteiro, uma string, um array ou um objeto, proporcionando um modelo de dados sem esquema.

Além disso, os bancos de dados de chave-valor são muito eficientes no armazenamento de dados distribuídos, mas não são adequados para cenários que requerem relações ou estruturas. Qualquer funcionalidade que requeira relações, estruturas ou ambas deve ser implementada na aplicação cliente que interage com o banco de dados de chave-valor.

Ora, como os valores são opacos para os bancos de dados, esses não podem lidar com consultas e indexações a nível de dados, podendo realizar consultas apenas por meio de chaves.

Um exemplo de banco de dados chave-valor é o Redis, que mantém os dados tanto na memória como no disco.

2.2.2 Colunar

Em bancos de dados colunares, o conjunto de dados consiste em várias linhas, cada uma identificada por uma chave de linha única, também conhecida como chave primária. Cada linha é composta por um conjunto de famílias de colunas, e diferentes linhas podem ter diferentes famílias de colunas. Semelhante aos bancos de dados de chave-valor, a chave de linha funciona como a chave, e o conjunto de famílias de colunas atua como o valor representado pela chave de linha. No entanto, cada família de colunas serve ainda como uma chave para uma ou mais colunas que contém, onde cada coluna consiste em um par nome-valor. O Cassandra oferece a funcionalidade adicional de supercolunas, que são criadas agrupando várias colunas.

Normalmente, os dados pertencentes a uma linha são armazenados juntos no mesmo nó do servidor. No entanto, o Cassandra pode distribuir uma única linha por vários nós de servidor usando chaves de partição compostas.

Nos bancos de dados de famílias de colunas, a configuração das famílias de colunas é tipicamente feita durante a inicialização. No entanto, a definição prévia de colunas não é necessária, oferecendo grande flexibilidade no armazenamento de qualquer tipo de dado.

Em geral, os bancos de dados de famílias de colunas fornecem capacidades de indexação e consulta mais poderosas do que os bancos de dados de chave-valor porque utilizam famílias de colunas e colunas além das chaves de linha.

Semelhante aos bancos de dados de chave-valor, qualquer lógica que exija relações deve ser implementada na aplicação cliente.

2.2.3 Documento

Bancos de dados orientados a documento armazenam registros como documentos autocontidos, em geral em JSON ou BSON. Cada documento possui um identificador e um conjunto de campos que podem variar entre documentos da mesma coleção, o que favorece esquemas flexíveis e agregados de leitura (*read-optimized*). O MongoDB é um exemplo popular: coleções de documentos, índices secundários e pipelines de agregação permitem consultas ricas sem exigir junções (*joins*) no servidor como no modelo relacional clássico.

Para o *Polyglot Import CSV*, o modelo documental é adequado a entidades com subestruturas aninhadas (por exemplo, um pedido com subdocumentos `buyer` e `product`), desde que o CSV de origem exponha colunas planas que o mapeamento JSON agrupe em subdocumentos.

2.2.4 Grafo

Bancos de dados em grafo representam dados como **nós** (entidades) e **arestas** (relacionamentos tipados), com propriedades em ambos. Consultas exploram caminhos e vizinhanças de forma natural, o que é útil para recomendação, detecção de fraude e redes sociais. O Neo4j utiliza a linguagem Cypher para `MERGE`/`CREATE` de nós e relacionamentos.

Na ferramenta proposta, o destino grafo exige que o usuário declare explicitamente quais colunas do CSV identificam cada rótulo de nó e quais colunas alimentam propriedades do relacionamento (por exemplo, comprador → vendedor com atributos do pedido).

2.3 Bancos de Dados Multimodelo

SGBDs **multimodelo** integram mais de um paradigma de dados no mesmo motor (por exemplo, documento + chave-valor + grafo). Isso reduz o número de produtos operados, mas concentra riscos de *lock-in* e de tuning em um único fornecedor (Ye et al., 2023).

A **persistência poliglota**, por outro lado, combina **vários SGBDs heterogêneos** escolhidos por adequação a cada caso de uso (Sadalage; Fowler, 2013). O *Polyglot Import CSV* segue esta segunda linha: o usuário direciona entidades e filtros para PostgreSQL, Redis, MongoDB, Cassandra e Neo4j conforme o padrão de acesso desejado.

3 TRABALHOS RELACIONADOS

3.1 Importação de arquivo CSV para um único SGBD

3.1.1 PostgreSQL

Primeiramente, você especifica a tabela com os nomes das colunas após a palavra-chave `COPY`. A ordem das colunas deve ser a mesma que aquelas no arquivo CSV.

```
COPY nome_da_tabela(col1, col2, col3)
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Caso o arquivo CSV contenha todas as colunas da tabela, você não precisa especificá-las explicitamente.

Segundo, você insere o caminho do arquivo CSV após a palavra-chave `FROM`. Como o formato do arquivo é CSV, você precisa especificar `DELIMITER`, bem como as cláusulas `CSV`.

Terceiro, especifique a palavra-chave `HEADER` para indicar que o arquivo CSV contém um cabeçalho. Quando o comando `COPY` importa dados, ele ignora o cabeçalho do arquivo.

```
COPY nome_da_tabela
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Observe que o arquivo deve ser lido diretamente pelo servidor PostgreSQL, não pela aplicação cliente. Portanto, ele deve ser acessível pela máquina do servidor PostgreSQL. Além disso, é necessário ter acesso de superusuário para executar com sucesso a instrução `COPY`.

3.1.2 Redis

A maneira preferida de importar dados em massa no Redis é gerar um arquivo de texto contendo o protocolo Redis, em formato bruto, para chamar os comandos necessários para inserir os dados requeridos.

Por exemplo, se precisar gerar um grande conjunto de dados com bilhões de chaves no formato “chaveN -> ValorN”, deve ser um arquivo contendo os seguintes comandos no formato de protocolo Redis:

```
SET Chave0 Valor0
SET Chave1 Valor1
...
SET ChaveN ValorN
```

Uma vez criado esse arquivo, a ação restante é alimentá-lo no Redis o mais rápido possível. No passado, a maneira de fazer isso era usar o netcat com o seguinte comando.

Nas versões 2.6 ou posteriores do Redis, a utilidade `redis-cli` suporta um novo modo chamado modo pipe que foi projetado para realizar o carregamento em massa.

Usando o modo pipe, o comando a ser executado parece o seguinte:

```
cat dados.txt | redis-cli --pipe
```

Isso produzirá uma saída semelhante a esta:

```
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 0, replies: 1000000
```

A utilidade `redis-cli` também garantirá redirecionar apenas os erros recebidos da instância Redis para a saída padrão.

3.1.3 MongoDB

Como a ferramenta `mongoimport` é fornecida oficialmente, o processo de importar dados em formato CSV é muito simples. É uma ferramenta poderosa e fácil de usar. Segue abaixo a sua sintaxe:

```
mongoimport --db nome_do_db --collection nome_da_colecao \
  --type csv --headerline --ignoreBlanks \
  --file caminho/do/arquivo.csv
```

- `--db nome_do_db`: define em qual banco de dados importar os dados.
- `--collection nome_da_colecao`: define o nome da nova coleção. Se esse parâmetro for omitido, o nome da coleção será o mesmo que o nome do arquivo CSV.
- `--type csv`: o tipo de arquivo é CSV.
- `--headerline`: o conteúdo da primeira linha do CSV será o nome de cada campo.
- `--ignoreBlanks`: esse parâmetro pode ignorar valores em branco no arquivo.
- `--file`: arquivo CSV a ser importado.

3.1.4 Cassandra

Importar dados CSV para o Cassandra usando `sstableloader` envolve vários passos. O `sstableloader` é uma ferramenta de linha de comando fornecida com o Apache Cassandra para carregar grandes volumes de dados eficientemente.

Certifique-se de que o arquivo CSV está no formato adequado para o Cassandra. Isso geralmente envolve garantir que os tipos de dados e as colunas correspondam ao esquema da tabela no Cassandra.

O `sstableloader` requer que os dados estejam em um formato específico chamado SSTable. Para converter o CSV em SSTables, você pode usar a ferramenta `cqlsh` fornecida com o Cassandra. Execute algo como:

```
cqlsh -e "COPY keyspace_name.table_name TO 'output_directory';"
```

Agora você pode usar o `sstableloader` para carregar os dados. O comando geralmente se parece com isso:

```
sstableloader -d <hostname> -u <username> -pw <password> \
  output_directory/keyspace_name/table_name
```

- `<hostname>`: O endereço do nó Cassandra.
- `<username>`: O nome de usuário, se a autenticação estiver habilitada.
- `<password>`: A senha correspondente.

Este comando moverá os dados do diretório de saída para o Cassandra usando o `sstableloader`.

3.1.5 Neo4j

Usar `neo4j-admin` para importar grandes conjuntos de dados no Neo4j envolve alguns passos. Esta ferramenta de linha de comando é projetada para importações em massa eficientes.

Certifique-se de que seus arquivos CSV estejam estruturados corretamente e sigam o formato necessário para nós e relacionamentos.

Crie um banco de dados não povoado:

```
neo4j-admin create-db --database=nome_do_seu_banco_de_dados \
  --from=diretorio_com_arquivos_csv
```

Substitua `nome_do_seu_banco_de_dados` pelo nome desejado para o novo banco de dados. A opção `--from` especifica o diretório onde estão localizados seus arquivos CSV.

Execute o comando de importação com `neo4j-admin`. O comando exato depende do seu modelo de dados, mas pode se parecer com isto:

```
neo4j-admin import --database=nome_do_seu_banco_de_dados \
  --mode=csv \
  --nodes:Rotulo1 nos.csv \
  --nodes:Rotulo2 nos2.csv \
  --relationships:TIPO_RELACIONAMENTO relacionamentos.csv
```

Substitua *Rotulo1*, *Rotulo2*, *nos.csv*, *nos2.csv*, *TIPO_RELACIONAMENTO* e *relacionamentos.csv* pelos seus rótulos específicos e nomes de arquivo.

3.2 Importação de arquivo CSV para SGBD multimodelo

Ferramentas comerciais e de código aberto tratam de **migração** ou **modelagem** entre tecnologias heterogêneas, embora raramente com o foco deste TCC (um único CSV largo + regras declarativas + cinco paradigmas).

- **ArangoDB** é um SGBD multimodelo nativo (documento/chave-valor/grafo) com `arangoimport` para CSV; a importação é para **um** motor multimodelo, não para cinco backends independentes.
- **`mongoimport` / `COPY` / `redis-cli --pipe` / `cqlsh` / `neo4j-admin`** (seção 3.1) cobrem importação **para um único** produto por vez.
- **dbcrossbar** ("dbcrossbar", 2024) copia dados tabulares entre PostgreSQL, BigQuery, armazenamento em nuvem e CSV, incluindo conversão de esquemas; a ênfase é *pipeline* de migração, não mapeamento semântico de entidades aninhadas com filtros por valor.
- **Hackolade** (Hackolade, 2024) oferece *polyglot data modeling* com engenharia de esquema para dezenas de alvos; trata-se de **modelagem visual** e geração de artefatos, e não de um utilitário CLI único para importar um CSV operacional com validação prévia de filtros.

3.3 Considerações sobre os trabalhos relacionados

A literatura sobre **metamodelos unificados** para SQL e NoSQL (por exemplo, U-Schema (Berlanga et al., 2021)) e sobre **evolução de esquema** em ambientes heterogêneos (Chillón et al., 2023) informa decisões de validação e de representação de entidades, mas não substitui uma ferramenta de *bulk import* orientada a CSV.

Sistemas **polystore** como o BigDAWG (Duggan et al., 2017) e middlewares de portabilidade em nuvem (Wang et al., 2020) tratam de consultas e movimentação de dados em ecossistemas complexos, enquanto o escopo deste TCC é propositalmente mais estreito: **importação declarativa** a partir de um arquivo CSV e configuração JSON, com validação estática e suporte inicial a cinco SGBDs representativos.

4 A PROPOSTA DA FERRAMENTA PolyglotImportCSV

4.1 Visão geral

A ferramenta **PolyglotImportCSV** (pacote Python `polyglot-import-csv`) lê um arquivo CSV e um arquivo de configuração JSON validado por um **JSON Schema** embutido. A CLI executa, em sequência:

1. Carregamento do CSV (como texto, para evitar erros de *parse* em campos com `+` em timestamps).
2. Inferência de *kinds* de coluna (inteiro, *float*, data/hora, texto) para validar operadores de filtro.
3. Validação cruzada: colunas referenciadas existem no CSV; relacionamentos PostgreSQL e Neo4j referenciam entidades conhecidas; chaves de partição Cassandra estão declaradas quando necessário.
4. Para cada backend configurado: aplicação de filtros (incluindo o operador `each` para particionar por valor distinto), materialização de entidades e importação (ou apenas resumo em `--dry-run`).

Comando principal:

```
python -m polyglotimportcsv caminho/dados.csv \
  --config caminho/import_config.json \
  [--dry-run] \
  [--create-schema / --no-create-schema] \
  [--only postgres,redis]
```

4.2 Formato de configuração

A raiz do JSON contém `version` e blocos opcionais `postgres`, `redis`, `mongodb`, `cassandra` e `neo4j`. Cada bloco declara `connection` (credenciais), `entities` (mapeamento coluna → metadados `is_key`, `db_column`, `db_type`) e `filters` (lista de predicados: `==`, `!=`, `>`, `<`, `>=`, `<=`, `in`, `not_in`, `each`). Entidades MongoDB podem conter `nested` para subdocumentos. Entidades Cassandra podem declarar `cassandra_partition` e `cassandra_cluster` para a chave primária.

O repositório inclui o exemplo `data/ecommerce/ecommerce_join.csv` (visão *larga* de e-commerce) e `data/ecommerce/import_config.json` alinhados ao cenário de persistência poliglota descrito na introdução.

4.3 Validação e execução

Erros de configuração levantam `BusinessException` antes de qualquer conexão. O modo `--dry-run` lista contagens por entidade sem contatar os SGBDs. O *driver* Apache Cassandra é importado de forma tardia para permitir `--dry-run` em ambientes onde a extensão C do *driver* não está disponível (por exemplo, versões recentes do Python).

Testes automatizados (`pytest`) cobrem filtros e um *smoke test* de validação + *dry-run*. O ficheiro `docker-compose.yml` na raiz sobe PostgreSQL, Redis, MongoDB, Cassandra e Neo4j para testes de integração manuais ou futuros testes de integração contínua.

5 ATIVIDADES FUTURAS

- **Interface gráfica desktop** (semestre seguinte) que monte o comando CLI e visualize o JSON de configuração.
- **Suporte a TSV/Excel** e a *chunked* processing para CSV maiores que a RAM.
- **Operadores de filtro adicionais** e políticas de coerção de tipos configuráveis.
- **Testes de integração** contra `docker compose` em CI.
- **Generalização** do núcleo de importação para novos conectores (mensageria, *data lakes*, outros SGBDs).

Referências

[] BERLANGA, Adrián *et al.* A Unified Metamodel for NoSQL and Relational Databases. **arXiv preprint arXiv:2105.06494**, 2021.

[] CHILLÓN, Alberto Hernández *et al.* Propagating Schema Changes to Code: An Approach Based on a Unified Data Model. *In*: 2023.

[] **dbcrossbar: copy tabular data between databases, CSV files, and cloud storage**. https://www.dbcrossbar.org, 2024.

[] DUGGAN, Jennie *et al.* BigDAWG: A Polystore System for Big Data Analytics. *In*: 2017.

[] FOWLER, Martin. **PolyglotPersistence**. Disponível em: https://martinfowler.com/bliki/PolyglotPersistence.html, 2011.

[] HACKOLADE. **Polyglot Data Modeling**. https://hackolade.com/polyglot-data-modeling.html, 2024.

[] HECHT, R.; JABLONSKI, S. NoSQL Evaluation: A Use Case Oriented Survey. *In*: 2011.

[] KHINE, Pwint Phyu; WANG, Zhaoshun. A Review of Polyglot Persistence in the Big Data World. **Information**, v. 10, n. 4, p. 141, 2019.

[] SADALAGE, Pramod J.; FOWLER, Martin. **NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence**. *[S.l.]*: Pearson Education, 2013.

[] WANG, L. *et al.* A Middleware for Polyglot Persistence and Data Portability of Big Data PaaS Cloud Applications. **Computers, Materials & Continua**, v. 65, n. 2, p. 1399–1416, 2020.

[] YE, Feng *et al.* A Benchmark for Performance Evaluation of a Multi-Model Database vs. Polyglot Persistence. **Journal of Database Management**, v. 34, n. 3, p. 1–20, 2023.